



Lumina Library Simple Team Guide

Read this guide before you change any code.

You have been invited to the Lumina GitHub repository. Accept the invitation, clone the project, check that the website opens, and then work only on your own module and branch.

Important rules

- Do not work directly on **main**.
- Work only on your assigned feature branch.
- Do not change the website theme, colors or common layout.
- If you have a UI idea, tell the project leader first.
- Do not upload passwords, .env files or database files to GitHub.
- Run the tests before you push your work.

Members and branches

Member	Management	Branch
1	Catalog	feature/catalog-management
2	Circulation	feature/circulation-management
3	Member Services	feature/member-services
4	Inventory & Assets	feature/inventory-management
5	Events & Programs	feature/event-management
6	Feedback & Complaints	feature/feedback-management

First setup on your laptop

1

Accept the invitation

Open the GitHub email or notification and click **Accept invitation**. Each member must do this. The project leader does not accept it for you.

2

Clone in IntelliJ

Open IntelliJ - Get from VCS. Paste the repository address and click Clone.

```
https://github.com/IT25102134/lumina-library.git
```

3

Open the project

Trust the project. Let IntelliJ load pom.xml as a Maven project. Set the Project SDK to Java 21.

4

Open your branch

Open the IntelliJ Terminal. Run the commands below. The Catalog branch is only an example.

```
git fetch origin
git switch -c feature/catalog-management --track origin/feature/catalog-management
git branch
```

Replace `feature/catalog-management` with your own branch name from page 1. The active branch has an asterisk (*) beside it.

Check that the website opens

For the first test, use the demo database:

```
.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=demo"
```

- Wait until IntelliJ says the application has started.
- The website should open at **http://localhost:8081**.
- If it does not open, type that address in your browser.
- Open your own management page and check that it works.
- Press Ctrl+C in the Terminal when you want to stop the website.

Do not start editing until the website works and you have checked that you are on your own branch.

Work and push from your branch

Do these steps every time you work. Catalog is shown as the example.

1**Open your branch**

Switch to your feature branch and download its newest changes.

```
git switch feature/catalog-management  
git pull origin feature/catalog-management
```

2**Make your changes**

Change only your management module. Do not change the shared UI theme.

3**Test the project**

Save your files and run the tests.

```
.\mvnw.cmd test
```

4**Check and commit**

Check the changed files, add them, and write a clear commit message.

```
git status  
git add .  
git commit -m "Catalog: describe your change"
```

5**Push your branch**

Upload your commit to the same feature branch.

```
git push origin feature/catalog-management
```

Change every Catalog branch name in this example to your own branch name. Never replace it with main.

Example commit messages

- Catalog: check ISBN before saving a book
- Circulation: stop renewal when a loan is overdue
- Events: add an attendance update button
- Feedback: show the complaint status

Do not use unclear messages such as update, done or changes.

After pushing: merge your own work

Leader approval is not required. Each member can create and merge their own Pull Request into main.

What each member must do

- 1 Open Pull Requests**
Open the Lumina repository on GitHub and click **Pull requests**.
- 2 Create the Pull Request**
Click **New pull request**. Choose **base: main** and choose your own feature branch as **compare**.
- 3 Check your files**
Open **Files changed**. Make sure you changed only your own module, did not change the UI theme, and did not upload passwords.
- 4 Create it**
Add a short description and click **Create pull request**.
- 5 Merge it**
If the tests pass and GitHub shows no conflict, click **Merge pull request**.
- 6 Confirm it**
Click **Confirm merge**. Your changes are now in main.

Do not push commits directly to main. Always use your feature branch and a Pull Request.

Check before you merge

- The website starts and your module works.
- The Maven tests pass.
- You changed only the files needed for your module.
- You did not change the shared UI theme.
- You did not upload a password, .env file or database file.
- If GitHub shows a conflict, stop and ask the group for help. Do not force the merge.

Update your laptop before the next task

After you merge the Pull Request, update main and then update your own branch. Catalog is shown as the example:

```
git fetch origin
git switch main
git pull origin main
git switch feature/catalog-management
git merge main
git push origin feature/catalog-management
```

Replace the Catalog branch with your own branch. After these commands finish, start your next change from your feature branch.

Set up SQL Server and SSMS

These steps create a separate local database on each member's laptop. Records made on one laptop will not appear on another laptop.

- 1 Install the software**
Install SQL Server Express or Developer Edition and SQL Server Management Studio (SSMS).
- 2 Open SSMS**
Connect to `localhost\SQLEXPRESS` with **Windows Authentication**.
- 3 Turn on SQL Server login mode**
Right-click the server - Properties - Security. Choose **SQL Server and Windows Authentication mode**. Restart the SQL Server service.
- 4 Turn on TCP/IP**
Open SQL Server Configuration Manager - SQL Server Network Configuration - Protocols for SQLEXPRESS. Enable TCP/IP.
- 5 Set port 1433**
Open TCP/IP Properties - IP Addresses - IPAll. Clear TCP Dynamic Ports. Set TCP Port to **1433**. Restart SQL Server (SQLEXPRESS).
- 6 Create the database**
In SSMS, connect with Windows Authentication. Open and run this complete file from the cloned project:

```
database/setup-sqlexpress.sql
```

The Results panel should say: Lumina SQL Server setup completed.

Test the new SQL login

SSMS box	Enter this value
Server type	Database Engine
Server name	localhost,1433
Authentication	SQL Server Authentication
Login	lumina_app
Password	Password written in setup-sqlexpress.sql
Certificate	Select Trust server certificate

Enter these details in the SSMS Connect to Server window. Do not type them inside a query tab.

Run the website with SQL Server

1

Open the run settings

In IntelliJ go to Run - Edit Configurations - LibraryApplication.

2

Set the profile

Enter sqlserver in Active profiles.

3

Add environment variables

Open Environment variables and add each line as a separate name and value.

```
SPRING_PROFILES_ACTIVE=sqlserver
DB_HOST=localhost
DB_PORT=1433
DB_NAME=lumina_library
DB_USERNAME=lumina_app
DB_PASSWORD=<password in setup-sqlexpress.sql>
SERVER_PORT=8081
OPEN_BROWSER=true
```

Do not select Store as project file. Keep the password only on your laptop. Never push it to GitHub.

4

Run the website

Click Run. Wait for Tomcat to start. The website should open at **http://localhost:8081**.

5

Make a test record

Add or edit one record from the website.

6

Check the record in SSMS

Open a new SSMS query and run:

```
USE lumina_library;
SELECT TOP 20 * FROM dbo.books ORDER BY id DESC;
```

If the team wants one shared database

- Only the leader's database laptop runs setup-sqlexpress.sql.
- Every member changes DB_HOST=localhost to the leader laptop's IPv4 address.
- The leader allows TCP port 1433 through Windows Firewall on the Private network.
- All laptops must be able to reach the leader's laptop.
- Do not open SQL Server port 1433 to the public internet.

Simple troubleshooting

- Login failed: check the username, password and SQL Server Authentication mode.
- Connection failed: check that SQL Server is running and TCP/IP port 1433 is enabled.
- Website uses the demo database: check that Active profiles is sqlserver.
- Port 8081 is busy: stop the other application or choose another SERVER_PORT.